Принципы разработки ПО

Keep It Simple, Stupid

KISS

- Функция должна быть небольшой.
- Одна функция решает одну проблему.
- При модификации кода в будущем не должно возникнуть трудностей.
- Система работает лучше, если она не усложняется без надобности.
- Не устанавливайте целую библиотеку ради одной функции.
- Не делай того, что не просят.
- Писать код необходимо надежно и «дубово».

You are not gonna need it

YAGNI

- Реализуйте только то, что нужно здесь и сейчас, а не “впрок”.
- Подчищайте ненужный код. Если что, найдёте в git.
- Не добавляйте новый функционал, если вас об этом не просят.

Don't Repeat Yourself

DRY

- Не копируйте код.
- Выносите общую логику.
- Прежде чем добавлять функционал, проверьте в проекте, может, он уже создан.
- Добавляйте константы.

Бритва Оккама

- Не плодите сущности без излишней необходимости.
- Начинайте с максимально простого кода, затем увеличивайте сложность по мере необходимости.

Big Design Up Front

BDUF

- Прежде чем переходить к реализации, убедитесь, что все продумано.
- Разработчик должен сначала завершить проектирование. После этого проект можно реализовать.
- Разделите требования на несколько этапов, определите приоритеты, начинайте с этапа с наивысшим приоритетом.
- Обсудите архитектуру проекта с командой и другими людьми, которые участвуют в проекте до старта.

Avoid Premature Optimization

APO

- Не стоит оптимизировать код до тех пор, пока в этом не возникнет необходимость.
- Ранняя оптимизация усложняет систему и может тратить ресурсы на нерешающие задачи.

SOLID

- Single responsibility principle (принцип единственной ответственности).
- Open-closed principle (принцип открытости/закрытости).
- Liskov substitution principle (принцип подстановки Барбары Лисков).
- Interface segregation principle (принцип разделения интерфейса).
- Dependency inversion principle (принцип инверсии зависимостей).

SOLID Single Responsibility Principle

Каждый класс или модуль должен иметь только одну причину для изменения. Это означает, что класс должен выполнять только одну конкретную функцию или задачу, что облегчает его понимание, тестирование и поддержку.

S**O**LID Open-Closed Principle

Данный принцип гласит, что программные сущности, такие как классы, модули и функции, должны быть открыты для расширения и закрыты для модификации. Другими словами, когда требуется изменить поведение системы, это должно происходить без изменения кода существующих компонентов, путем добавления новых компонентов или расширения существующих.

S**O**LID Open-Closed Principle

Данный принцип гласит, что программные сущности, такие как классы, модули и функции, должны быть открыты для расширения и закрыты для модификации. Другими словами, когда требуется изменить поведение системы, это должно происходить без изменения кода существующих компонентов, путем добавления новых компонентов или расширения существующих.

SOLID Liskov Substitution Principle

Данный принцип акцентирует важность того, чтобы объекты подклассов могут быть использованы в коде программы вместо объектов их базовых классов, не вызывая непредвиденных проблем или нарушений функциональности. Объекты подклассов должны быть подставляемы вместо объектов базовых классов без нарушения корректности выполнения программы.

SOLID Interface Segregation Principle

Данный принцип уделяет внимание разделению больших, общих интерфейсов на более мелкие и специфичные, чтобы клиенты могли зависеть только от тех методов, которые им действительно нужны.

SOLI**D** Dependency Inversion Principle

Данный принцип заключается в том, что зависимости между модулями и компонентами программы должны быть организованы таким образом, чтобы более низкоуровневые компоненты зависели от абстракций, а не наоборот.